

Pushing the Limits of Novel View Synthesis in Soccer Scenes Through Ensembles

Fabian Perez

Juan Vanegas

Christian Orduz

Hoover Rueda-Chacón*

Department of Computer Science, Universidad Industrial de Santander, Colombia

{perez2258059, juan2221931, christian.orduz}@correo.uis.edu.co

* hfarueda@uis.edu.co

Abstract

We, the Hands-On Computer Vision team, propose a novel view synthesis pipeline for soccer scenes. We enhance the baseline along two axes: monocular depth priors combined with an improved appearance module to guide geometric reconstruction, and an ensemble that aggregates multiple renderings to reduce variance under extreme novel viewpoints. Additionally, we apply a final color-matching post-processing step. Our code will be publicly available at: <https://github.com/cvail-research/soccernet-nvs-2026>.

1. Proposed Method

1.1. Overview

Our method builds upon the 3D Gaussian Splatting [1] framework, using the open-source *gsplat* [2] library from Nerfstudio as our backbone, and enhances it along two complementary axes. First, we incorporate monocular depth priors to guide the geometric reconstruction. Second, we use a model ensembling strategy that aggregates the predictions of several independently trained renderings, each contributing its respective strengths.

1.2. Architecture

1.2.1. Appearance Compensation Module

We enhance the appearance module in *gsplat* [2] by incorporating feature normalization, regularization, and a residual learning formulation. Specifically, for a camera c and a Gaussian i , let $\mathbf{e}_c \in \mathbb{R}^{d_e}$ denote the camera-specific embedding, $\mathbf{f}_i \in \mathbb{R}^{d_f}$ the per-Gaussian latent feature vector, and $\mathbf{SH}_\ell(\mathbf{d}_{c,i}) \in \mathbb{R}^K$ the spherical harmonics basis of degree ℓ evaluated at the viewing direction $\mathbf{d}_{c,i} \in \mathbb{R}^3$. We first apply stochastic regularization to the camera embedding, $\tilde{\mathbf{e}}_c = \text{Dropout}(\mathbf{e}_c)$ and concatenate the regularized embedding with the remaining inputs to form the input:

$$\mathbf{h}_{c,i} = [\tilde{\mathbf{e}}_c; \mathbf{f}_i; \mathbf{SH}_\ell(\mathbf{d}_{c,i})] \in \mathbb{R}^D, \quad (1)$$

where $D = d_e + d_f + K$. Finally, we apply layer normalization to obtain the input passed to the appearance MLP, $\tilde{\mathbf{h}}_{c,i} = \text{LN}(\mathbf{h}_{c,i})$. The appearance network is modeled as a multi-layer perceptron g_θ :

$$\Delta \mathbf{c}_{c,i} = \tanh(g_\theta(\tilde{\mathbf{h}}_{c,i})) \odot \mathbf{s}, \quad (2)$$

where $\Delta \mathbf{c}_{c,i} \in \mathbb{R}^3$ is a residual color correction, $\mathbf{s} \in \mathbb{R}^3$ is a learnable per-channel scaling, and $\odot$ denotes element-wise multiplication. Let $\mathbf{C}_{c,i}^{\text{base}} \in \mathbb{R}^3$ denote the learnable per-Gaussian colors. The final color $\mathbf{C}_{c,i} \in [0, 1]^3$ is obtained by adding the residual correction $\mathbf{C}_{c,i} = \mathbf{C}_{c,i}^{\text{base}} + \Delta \mathbf{c}_{c,i}$.

1.2.2. Depth Prior Integration

Instead of relying solely on Structure-from-Motion (SfM) initialization from COLMAP [3], we employ relative monocular depth predictions obtained from Depth Anything V3 $\mathcal{D}(\cdot)$ (DA3-Large 0.36B parameters) [4]. These depth maps provide dense geometric priors to guide the reconstructions. For each training image $\mathbf{I} \in \mathbb{R}^{H \times W \times 3}$ we use $\mathcal{D}(\cdot)$ to produce dense depth maps $\mathbf{Y} = \mathcal{D}(\mathbf{I}) \in \mathbb{R}^{H \times W}$.

1.2.3. Loss Function

Given the rendered depth map $\hat{\mathbf{Y}} \in \mathbb{R}^{H \times W}$ and ground truth $\mathbf{Y} \in \mathbb{R}^{H \times W}$, we first compute an optimal scale s and shift t such that $\hat{\mathbf{Y}}^* = s\hat{\mathbf{Y}} + t$ where (s, t) are obtained by minimizing the squared error between $\mathbf{Y}$ and $\hat{\mathbf{Y}}^*$. The depth loss is then defined as:

$$\mathcal{L}_{\text{depth}} = \frac{1}{2} \left\| \mathbf{Y} - \hat{\mathbf{Y}}^* \right\|_2^2. \quad (3)$$

We also introduce a gradient-based regularization term on the residual $\mathbf{R} = \mathbf{Y} - \hat{\mathbf{Y}}^*$: The gradient loss is defined as:

$$\mathcal{L}_{\text{grad}} = (\|\nabla_x \mathbf{R}\|_1 + \|\nabla_y \mathbf{R}\|_1), \quad (4)$$

where ∇_x and ∇_y denote discrete spatial derivatives along the horizontal and vertical directions, respectively.

We adopt the standard photometric loss $\mathcal{L}_{\text{photo}}$ from 3D Gaussian Splatting[1]. The total loss is:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{photo}} + \lambda_d (\mathcal{L}_{\text{depth}} + \lambda_g \mathcal{L}_{\text{grad}}), \quad (5)$$

where $\mathcal{L}_{\text{depth}}$ enforces alignment with the monocular depth, and $\mathcal{L}_{\text{grad}}$ promotes consistency of geometric structures.

1.3. Ensemble

We observe that different reconstruction models evaluated in extreme novel views exhibit complementary strengths: our 3DGS model faithfully reconstructs the largely static stadium background, whereas a model trained under the Triangle Splatting [5] formulation produces noticeably sharper renderings of the players and the ball. To leverage the best of both, we combine their predictions through a weighted ensemble in which the per-pixel weights are guided by the semantic content of each rendered view. To identify these regions, given a rendered novel view $\hat{\mathbf{X}} \in \mathbb{R}^{H \times W \times 3}$ produced by a trained model, we run inference with a pre-trained segmentation network f_θ (SAM3 [6]) and obtain a binary mask

$$\mathcal{M} = f_\theta(\hat{\mathbf{X}}) \in \{0, 1\}^{H \times W \times |\mathcal{R}|}, \quad (6)$$

that isolates the foreground classes $\mathcal{R} = \{\text{ball}, \text{player}_1, \text{player}_2, \dots, \text{player}_{N_p}\}$ from the rest of the scene. We then collapse $\mathcal{M}$ across the class dimension into a single binary foreground indicator

$$\mathbf{M}_{h,w} = \max_{r \in \{1, \dots, |\mathcal{R}|\}} \mathcal{M}_{h,w,r} \in \{0, 1\}^{H \times W}, \quad (7)$$

beyond combining different model families, we further reduce the prediction error in image space by averaging multiple independently trained renderings. For N renderings $\{\hat{\mathbf{X}}_n\}_{n=1}^N$ whose errors with respect to the ground-truth view $\mathbf{X}^*$ are zero-mean and independent with variance σ^2 , the per-pixel error variance of their mean satisfies:

$$\text{Var} \left[\frac{1}{N} \sum_{n=1}^N \hat{\mathbf{X}}_n - \mathbf{X}^* \right] = \frac{\sigma^2}{N}, \quad (8)$$

This is especially relevant in the extreme novel-view regime, where each individual rendering exhibits its largest errors. Concretely, we train $N_g = 5$ 3DGS renderings $\{\hat{\mathbf{X}}_n^{(g)}\}_{n=1}^{N_g}$ together with a single Triangle Splatting rendering $\hat{\mathbf{X}}^{(t)}$, and we denote the 3DGS average by

$$\bar{\mathbf{X}}^{(g)} = \frac{1}{N_g} \sum_{n=1}^{N_g} \hat{\mathbf{X}}_n^{(g)}. \quad (9)$$

We then assign a per-pixel weight map to each rendering. The Triangle Splatting rendering is weighted only on foreground pixels with $w_t = 0.5$ we set $\mathbf{w}^{(t)} = w_t \cdot \mathbf{M}$, i.e. it receives a weight of 0.5 where the mask is active and 0 elsewhere, while the 3DGS average receives the complementary weight $\mathbf{w}^{(g)} = \mathbf{1} - \mathbf{w}^{(t)}$, with $\mathbf{1}$ denoting the all-ones $H \times W$ map. The final ensembled rendering is

$$\hat{\mathbf{X}}_{\text{ens}} = \mathbf{w}^{(t)} \odot \hat{\mathbf{X}}^{(t)} + \mathbf{w}^{(g)} \odot \bar{\mathbf{X}}^{(g)}, \quad (10)$$

Scene	gsplat		Ours	
	PSNR $\uparrow$	SSIM $\uparrow$	PSNR $\uparrow$	SSIM $\uparrow$
Scene 1	30.7053	0.8229	29.8623	0.8146
Scene 2	28.2402	0.8043	30.1390	0.8414
Scene 3	26.5945	0.7814	29.1802	0.8417
Scene 4	28.5718	0.8130	28.6485	0.8246
Scene 5	28.6487	0.7864	29.0613	0.8010
Average	28.5521	0.8016	29.3783	0.8247

Table 1. Comparison between gsplat baseline and our method in a synthetic validation set. Best results per scene are shown in bold.

2. Experimental Results

Since the official challenge split is held out, we build a synthetic validation set by holding out 20% of the training images per scene. To mirror the viewpoint distribution of the challenge split, half of the held-out frames correspond to broadcast-style views and the other half to extreme close-up views.

Implementation details. All 3DGS models are trained for 80k iterations with AdamW on a single NVIDIA RTX PRO 6000 Blackwell GPU using the modified gsplat [2] framework, we set the loss weights to $\lambda_d = 10^{-3}$ and $\lambda_g = 0.5$, and keep all other hyperparameters at their default values. Each model is trained in 1 hour. The Triangle Splatting model is trained with the public official code [5].

Table 1 reports per-scene PSNR and SSIM on our validation split. Our method consistently outperforms the *gsplat* baseline, winning 4 out of 5 scenes on both metrics and raising the averages by +0.83 dB in PSNR (28.55 $\rightarrow$ 29.38) and +0.023 in SSIM (0.802 $\rightarrow$ 0.825). The largest gains concentrate on Scene 2 (+1.90 dB) and Scene 3 (+2.59 dB), where the baseline geometry is weakest, indicating that our depth-prior supervision and appearance-compensation module are most beneficial when SfM provides limited geometric constraints. The only exception is Scene 1, where the baseline retains a small advantage.

For the final challenge submission, we re-train each model on the full set of training views and apply a per-rendering color-matching post-processing step. Specifically, we transfer the color statistics of the training views onto every rendered image using the Multi-Variate Gaussian Distribution (MVGD) transfer [7], an analytical color-matching method that aligns the per-channel mean and covariance of the rendering with those of a reference image drawn from the training views. This step mitigates the residual color drift that 3DGS-based pipelines tend to exhibit under extreme novel viewpoints, where the appearance module extrapolates to camera embeddings far from the training distribution.

References

- [1] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, “3d gaussian splatting for real-time radiance field rendering,” *ACM Transactions on Graphics*, vol. 42, no. 4, July 2023. [Online]. Available: <https://repo-sam.inria.fr/fungraph/3d-gaussian-splatting/>
- [2] V. Ye, R. Li, J. Kerr, M. Turkulainen, B. Yi, Z. Pan, O. Seiskari, J. Ye, J. Hu, M. Tancik, and A. Kanazawa, “gsplat: An open-source library for gaussian splatting,” *Journal of Machine Learning Research*, vol. 26, no. 34, pp. 1–17, 2025.
- [3] J. L. Schönberger and J.-M. Frahm, “Structure-from-motion revisited,” in *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [4] H. Lin, S. Chen, J. H. Liew, D. Y. Chen, Z. Li, G. Shi, J. Feng, and B. Kang, “Depth anything 3: recovering the visual space from any views,” *arXiv preprint arXiv:2511.10647*, 2025.
- [5] J. Held, R. Vandeghen, A. Deliege, A. Hamdi, D. Rebain, S. Giancola, A. Cioppa, A. Vedaldi, B. Ghanem, A. Tagliasacchi *et al.*, “Triangle splatting for real-time radiance field rendering,” in *Thirteenth International Conference on 3D Vision*, 2025.
- [6] N. Carion, L. Gustafson, Y.-T. Hu, S. Debnath, R. Hu, D. Suris, C. Ryali, K. V. Alwala, H. Khedr, A. Huang *et al.*, “Sam 3: Segment anything with concepts,” *arXiv preprint arXiv:2511.16719*, 2025.
- [7] C. Hahne and A. Aggoun, “Plenopticam v1.0: A light-field imaging framework,” *IEEE Transactions on Image Processing*, vol. 30, pp. 6757–6771, 2021.